

SwarmOps Cloud

C4 Architecture Model

September 2026

Abstract

This document describes the architecture of SwarmOps Cloud with the C4 model. The C4 model presents software architecture as a hierarchy of four levels of abstraction — system context, containers, components and code — supplemented by dynamic and deployment diagrams. Each diagram below is rendered from a C4-PlantUML source file kept in the repository, so the diagrams change together with the code they describe. For every level, a catalogue table states each element's responsibility and technology.

Contents

1	The C4 model	1
2	Level 1 — System context	2
3	Level 2 — Containers	4
4	Level 3 — Components	4
4.1	SwarmOps Core	4
4.2	Commerce service	8
5	Level 4 — Code	9
6	Dynamic view — from order to running application	9
7	Deployment view	11
7.1	Intended production deployment	11
7.2	Environment used for verification	12
8	References	12

List of Figures

1	System context diagram	3
2	Container diagram	5
3	Components of SwarmOps Core	6
4	Components of the commerce service	8
5	Principal types of commerce, the queue and storage	10
6	Dynamic diagram: ordering a plan until the application runs	10
7	Intended production deployment	13
8	Environment used for verification	14

List of Tables

1	Elements of the system context	2
2	Containers	4
3	Components of SwarmOps Core	7
4	Components of the commerce service	8
5	Steps of the order-to-deploy scenario	11
6	Nodes of the production deployment	11

1 The C4 model

The C4 model, introduced by Simon Brown [1], describes a software system at four levels of zoom:

1. **System context** — the system as a box, the people who use it and the other systems it depends on.
2. **Containers** — the separately running or deployable units inside the system (applications, databases, services) and how they communicate.
3. **Components** — the major structural building blocks inside one container and their responsibilities.
4. **Code** — the classes, types and interfaces that implement a component.

Two supplementary diagram types are used here. *Dynamic diagrams* show how elements collaborate at run time for one scenario. *Deployment diagrams* show how containers map onto infrastructure.

Notation. People are drawn as person shapes, the system under discussion and its parts in blue, and external systems in grey. Databases are drawn as cylinders. Arrows are one-directional and labelled with the intent of the interaction and, where useful, the protocol. Each diagram includes a legend.

Sources. The diagrams are kept in `docs/cloud/diagrams/*.puml` and use the C4-PlantUML library bundled with PlantUML. The documentation build renders them to PNG and SVG.

2 Level 1 — System context

Figure 1 shows SwarmOps Cloud in its environment.

Table 1: Elements of the system context

Element	Kind	Responsibility
Customer	Person	Buys hosting plans, tops up a prepaid wallet, follows orders and projects, reads invoices and opens support tickets.
Administrator	Person	Confirms or rejects orders; manages plans, wallets, billing and tickets; operates servers, the gateway and applications.
Application visitor	External person	Uses an application that a customer deployed.
SwarmOps Cloud	Software system	The system described here: storefront, administration console and SwarmOps Core with its database.
Docker Swarm cluster	External system	The servers that run the machine agent, Docker Engine, the Traefik gateway and customer applications.
Container registry	External system	Holds the images customers order.
Git providers	External system	Repositories read by the source-to-deploy feature SwarmOps already had.
DNS providers	External system	Cloudflare and ArvanCloud zones used for records and DNS-01 certificate challenges.
ACME certificate authority	External system	Issues TLS certificates to the gateway.

Boundary decisions. The Swarm cluster is outside the system boundary because customers' workloads run there and it is operated through a narrow interface: fixed-shape operations sent to the machine agent. A payment gateway does not appear: top-ups are simulated in this release (Proposal, section 6).

SwarmOps Cloud — System Context (C4 level 1)

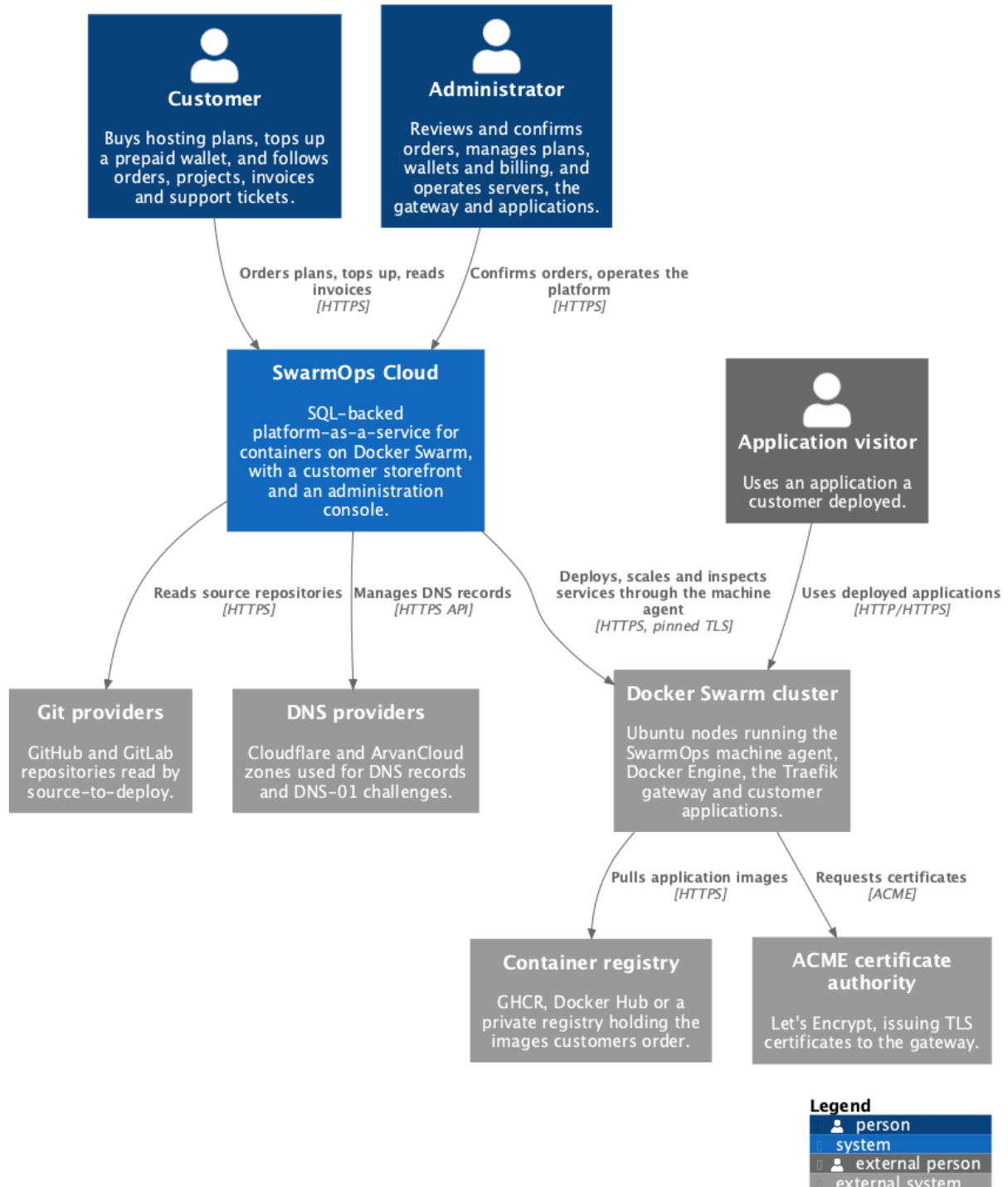


Figure 1: System context diagram

3 Level 2 — Containers

Figure 2 opens the SwarmOps Cloud box.

Table 2: Containers

Container	Technology	Responsibility
Storefront	React 19, TypeScript, Vite; served at <code>/store</code>	Customer single-page application in English and Persian: plans, checkout, wallet, orders, projects, invoices, support.
Administration console	React 19, TypeScript, Vite; served at <code>/</code>	Operator single-page application: the Sales area plus servers, gateway, applications, runs and audit.
SwarmOps Core	Go 1.26, <code>net/http</code>	Serves both applications and their JSON APIs; authenticates users; runs the command worker and the billing job.
Controller database	MariaDB 11.4 or MySQL 8.4	Holds all state: 57 tables and 3 views across nine migrations. Secret columns are encrypted.
Machine agent	Go, <code>systemd</code> service on each node	Performs Docker and Swarm operations requested by Core.
Traefik gateway	Traefik v3.6, Swarm service	Routes HTTP traffic to applications and obtains certificates.
Customer applications	Docker Swarm services	One stack per project, limited to its plan's CPU and memory.

Why the worker and billing job are not separate containers. Both run inside the Core process. Only the *active* core may execute commands or charge wallets (constraint C-3 of the SRS). Keeping them in the same process means the check that decides whether this core is active is the same check the API uses, so a standby core cannot bill or deploy by accident. They appear as components at level 3.

Why one database for both platform and commerce. Confirming an order must charge the wallet and queue the deployment atomically. Because the command queue and the wallet ledger live in the same database, a single local transaction achieves this without a distributed protocol. Section 6 describes the pattern.

4 Level 3 — Components

4.1 SwarmOps Core

Figure 3 shows the components inside SwarmOps Core.

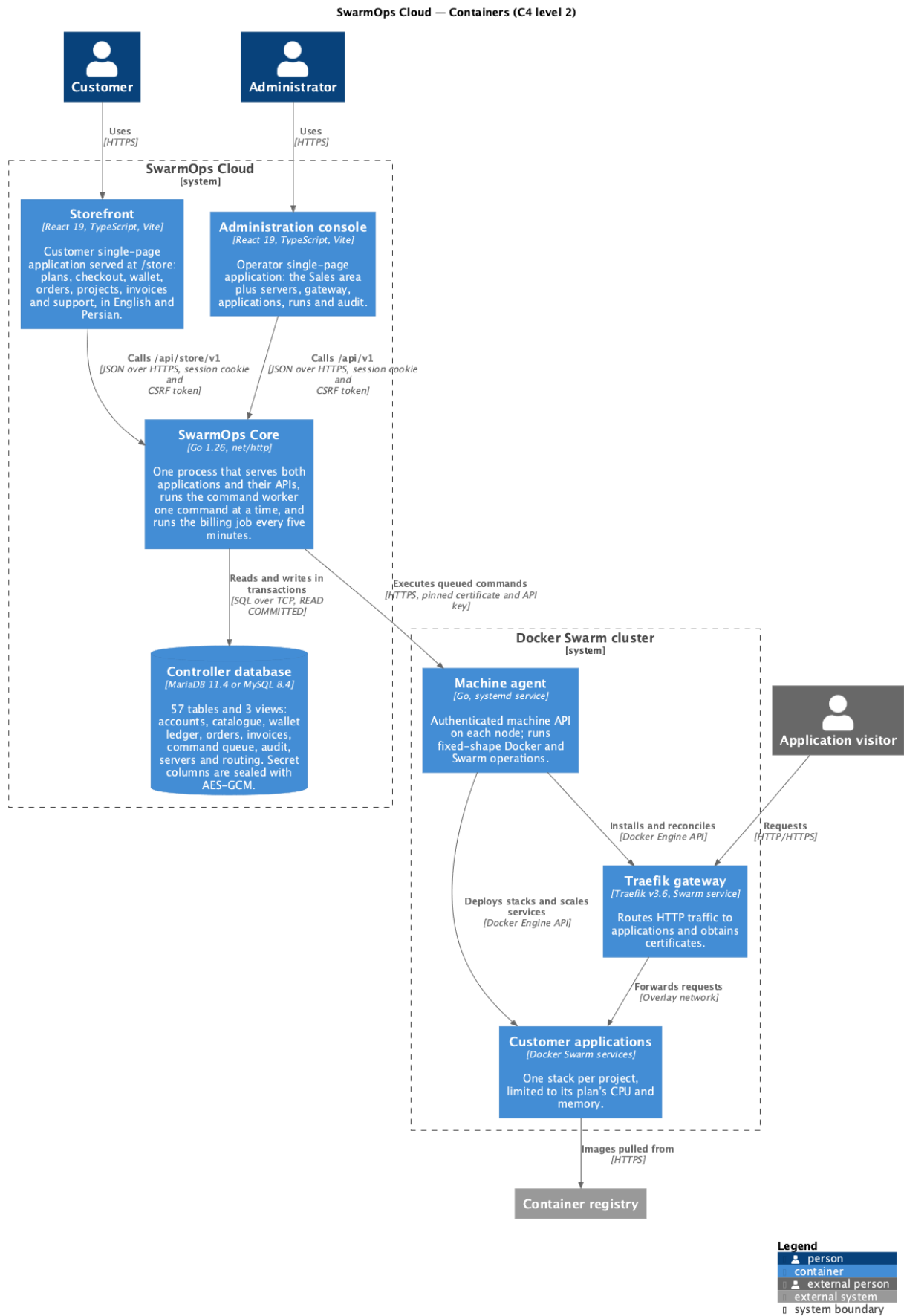


Figure 2: Container diagram

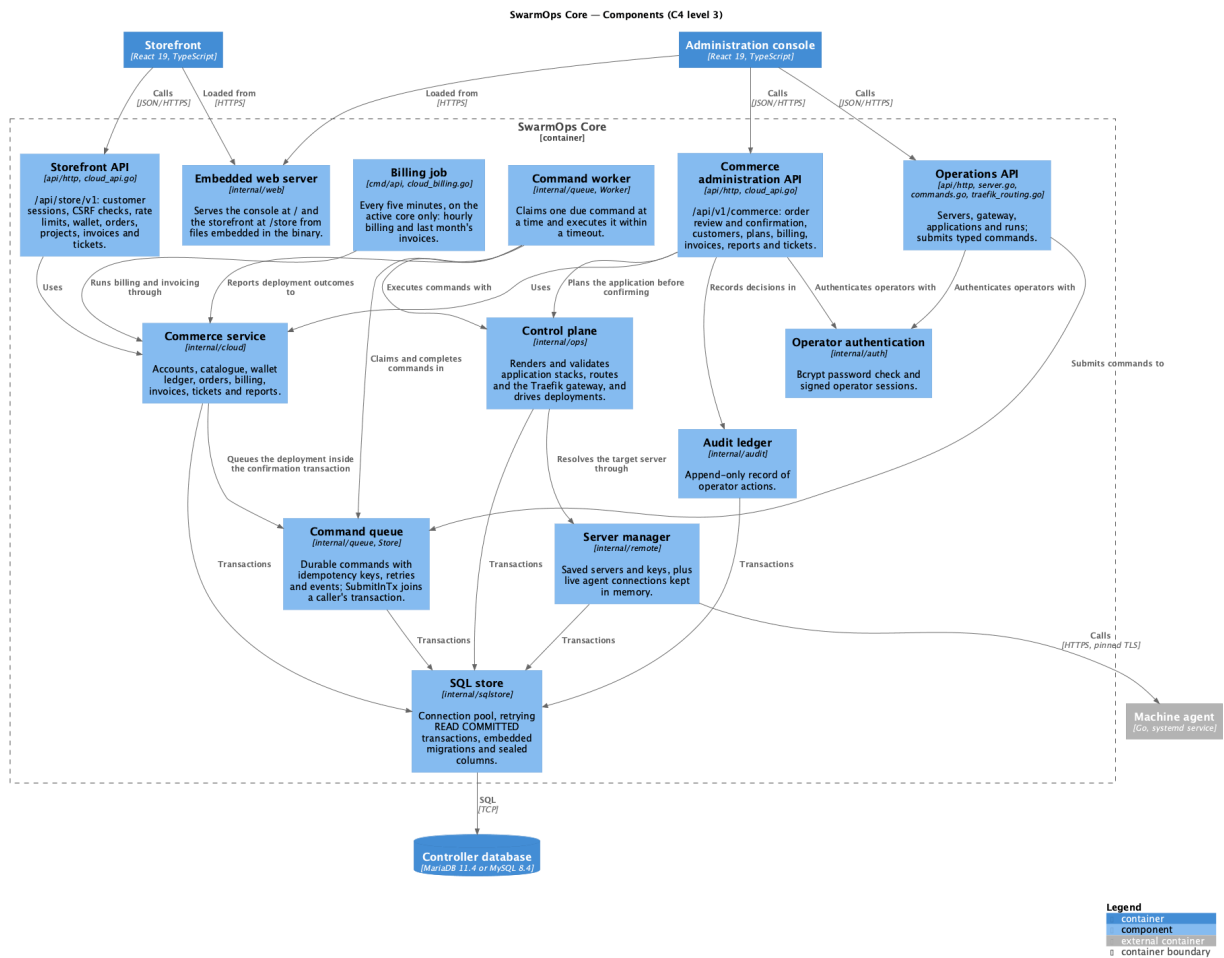


Figure 3: Components of SwarmOps Core

Table 3: Components of SwarmOps Core

Component	Code	Responsibility
Embedded web server	<code>internal/web</code>	Serves the console and the storefront from files embedded in the binary.
Storefront API	<code>api/http/cloud_api.go</code>	<code>/api/store/v1</code> : customer session cookie, CSRF check, sign-in rate limiting, and the customer's wallet, orders, projects, invoices and tickets.
Commerce administration API	<code>api/http/cloud_api.go</code>	<code>/api/v1/commerce</code> : order review and confirmation, customers, plans, billing, invoices, reports and tickets, behind operator authentication.
Operations API	<code>api/http (server.go, commands.go, traefik_routing.go and others)</code>	The existing SwarmOps operations; each change is submitted as a command.
Operator authentication	<code>internal/auth</code>	Checks the operator password and signs operator sessions.
Commerce service	<code>internal/cloud</code>	Business rules for accounts, catalogue, wallet, orders, billing, invoices, tickets and reports.
Billing job	<code>cmd/api/cloud_billing.go</code>	Every five minutes on the active core: hourly billing, then last month's invoices.
Command queue	<code>internal/queue (Store)</code>	Durable commands with idempotency keys, retries, events and outputs; <code>SubmitInTx</code> joins a caller's transaction.
Command worker	<code>internal/queue (Worker)</code>	Claims one due command at a time and executes it within a timeout; reports transitions.
Control plane	<code>internal/ops</code>	Renders and validates application stacks, routes and the gateway; drives deployments.
Server manager	<code>internal/remote</code>	Saved servers and keys in SQL; live agent connections in memory.
Audit ledger	<code>internal/audit</code>	Append-only record of operator actions.
SQL store	<code>internal/sqlstore</code>	Connection pool, retrying transactions, embedded migrations and sealed columns.

4.2 Commerce service

Figure 4 opens the commerce service. Each component corresponds to one source file in `internal/cloud`, and all share one `Service` value.

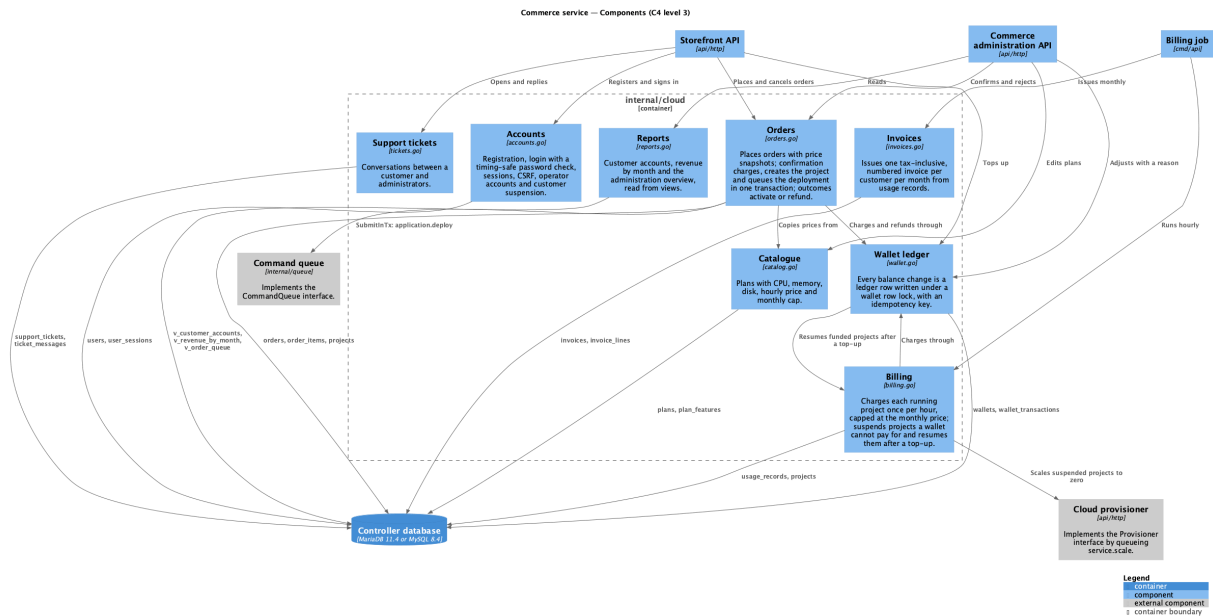


Figure 4: Components of the commerce service

Table 4: Components of the commerce service

Component	File	Responsibility	Tables
Accounts	<code>accounts.go</code>	Registration, sign-in with a timing-safe check, sessions and CSRF tokens, operator accounts, customer suspension.	<code>users</code> , <code>user_sessions</code>
Catalogue	<code>catalog.go</code>	Plans and their features.	<code>plans</code> , <code>plan_features</code> , <code>product_categories</code>
Wallet ledger	<code>wallet.go</code>	Applies every balance change as a ledger row under a wallet row lock, with an idempotency key.	<code>wallets</code> , <code>wallet_transactions</code>
Orders	<code>orders.go</code>	Places orders with price snapshots; confirmation charges, creates the project and queues the deployment in one transaction; outcomes activate or refund.	<code>orders</code> , <code>order_items</code> , <code>projects</code> , <code>usage_records</code>

Component	File	Responsibility	Tables
Billing	<code>billing.go</code>	Hourly charges capped monthly; suspension and resumption through the provisioner.	<code>usage_records</code> , <code>projects</code>
Invoices	<code>invoices.go</code>	Monthly, tax-inclusive, numbered invoices from usage records.	<code>invoices</code> , <code>invoice_lines</code>
Support tickets	<code>tickets.go</code>	Customer-administrator conversations.	<code>support_tickets</code> , <code>ticket_messages</code>
Reports	<code>reports.go</code>	Customer accounts, revenue by month and the overview, read from views.	<code>v_customer_accounts</code> , <code>v_revenue_by_month</code> , <code>v_order_queue</code>

The service depends on two interfaces rather than on concrete packages. **CommandQueue** is implemented by the command queue's **Store**. **Provisioner** is implemented in the HTTP layer by queueing a `service.scale` command. This keeps `internal/cloud` testable with fakes and free of HTTP and Swarm details.

5 Level 4 — Code

Figure 5 shows the principal types behind orders, the wallet and the command queue.

Four points in this diagram carry the design:

- **Service.applyTx** is the only code that changes a balance. It takes an unexported `ledgerEntry`, locks the wallet row with `SELECT ... FOR UPDATE`, and returns the existing transaction if the idempotency key was already used. Otherwise it inserts the ledger row and updates the cached balance. A `CHECK` constraint on `wallets.balance_rial >= 0` rejects any overdraft the code might miss.
- **Service.ConfirmOrder** runs inside `sqlstore.DB.WithTx`. It calls `applyTx` and inserts the project and usage rows. It then calls `CommandQueue.SubmitInTx` with the same `*sql.Tx`, so the deployment command is committed with the charge.
- **Service.OnCommandTransition** receives the final state of an `application.deploy` command. It activates the project and order, or fails them, deletes the refunded usage row and refunds the charge with a fixed idempotency key.
- **queue.Worker** claims one `Record` at a time with `Store.ClaimDue`, which locks the next due row with `FOR UPDATE SKIP LOCKED`. It executes the command within `ExecutionTimeout`, and `Store.Fail` schedules a retry with backoff unless the error is permanent.

6 Dynamic view — from order to running application

Figure 6 shows the scenario that ties the system together. The numbered steps are explained in the table.

Figure 5: Principal types of commerce, the queue and storage

```

sequenceDiagram
    participant Customer as «person» Customer
    participant Administrator as «person» Administrator
    participant Storefront as «container» Storefront [React]
    participant AdminConsole as «container» Administration console [React]
    participant SwarmOpsCore as «container» SwarmOps Core [Go]
    participant ControllerDB as «container» Controller database [MariaDb or MySQL]
    participant MachineAgent as «container» Machine agent [Go]
    participant CustomerApp as «container» Customer application [Swarm service]

    Customer->>Storefront: 1: Chooses a plan and enters the application name, image and port
    Storefront->>SwarmOpsCore: 2: POST /api/store/v1/orders [JSON/HTTPS]
    Administrator->>AdminConsole: 4: Reviews the order, picks a Swarm manager and types CONFIRM_ORDER_n
    AdminConsole->>SwarmOpsCore: 5: POST /api/v1/commerce/orders/{id}/confirm [JSON/HTTPS]
    SwarmOpsCore->>ControllerDB: 3: Inserts the order and its item with the prices at this moment; status pending [SQL]
    SwarmOpsCore->>ControllerDB: 7: One transaction: lock the wallet, check the reserve, charge the first hour, create the project and usage row, insert the application.deploy command [SQL]
    SwarmOpsCore->>ControllerDB: 8: The worker claims the due command [SQL, FOR UPDATE SKIP LOCKED]
    SwarmOpsCore->>MachineAgent: 6: Plans the application against the chosen server [HTTPS]
    MachineAgent->>MachineAgent: 9: Deploys the stack and waits for it to converge [HTTPS]
    MachineAgent->>CustomerApp: 10: Creates the service; Swarm checks GET /healthz [Docker Engine API]
    CustomerApp->>SwarmOpsCore: 11: Succeeded: project and order become active. Failed: both become failed, the usage row is deleted and the hour is refunded [SQL]
  
```

The diagram illustrates the lifecycle of a Docker Swarm application, involving a Customer, an Administrator, and several system components (Storefront, Administration console, SwarmOps Core, Controller database, Machine agent, and Customer application). The process is divided into 11 numbered steps:

- Customer** chooses a plan and enters the application name, image, and port.
- Storefront** (React) sends a `POST /api/store/v1/orders` request to **SwarmOps Core** (Go).
- Administrator** reviews the order and picks a Swarm manager, then types `CONFIRM_ORDER_n`.
- Administration console** (React) sends a `POST /api/v1/commerce/orders/{id}/confirm` request to **SwarmOps Core** (Go).
- SwarmOps Core** inserts the order and its items with prices at the current moment, with a status of pending, into the **Controller database** (MariaDb or MySQL).
- SwarmOps Core** performs a transaction: locks the wallet, checks the reserve, charges the first hour, creates the project and usage row, and inserts the `application.deploy` command into the **Controller database**.
- SwarmOps Core** claims the due command from the **Controller database** using `SQL, FOR UPDATE SKIP LOCKED`.
- SwarmOps Core** plans the application against the chosen server using `HTTPS`.
- Machine agent** (Go) deploys the stack and waits for it to converge using `HTTPS`.
- Machine agent** creates the service; Swarm checks `GET /healthz` using the `Docker Engine API`.
- Customer application** (Swarm service) sends a request to **SwarmOps Core** (Go) indicating success or failure. Success means the project and order become active; failure means both become failed, the usage row is deleted, and the hour is refunded.

Figure 6: Dynamic diagram: ordering a plan until the application runs

Table 5: Steps of the order-to-deploy scenario

Step	What happens	Guarantee
1–3	The customer orders; Core stores the order and its item with the current prices.	Later price changes do not affect the order.
4–5	The administrator reviews the order, picks a Swarm manager and confirms with a typed phrase.	Accidental confirmation is prevented.
6	Core plans the application against the chosen server before touching money.	An application that cannot run is refused without a charge.
7	One transaction locks the wallet, checks the 24-hour reserve, charges the first hour, creates the project and usage row, and inserts the deployment command.	Charge and deployment request commit together or not at all.
8	The command worker claims the command.	Only the active core executes; a crash before completion leaves the command to be retried.
9–10	The agent deploys the stack; Swarm waits for the health check.	The plan’s CPU and memory limits apply.
11	The outcome activates the order and project, or fails them and refunds the hour.	The refund happens exactly once.
12	The customer sees the project and its charges.	—

This scenario was run end to end during verification. Order 1 failed because the gateway was not yet installed. Order 2 failed because its image did not answer the health check. Both were refunded. Order 3 became active (Test Plan and Report, section 5).

7 Deployment view

7.1 Intended production deployment

Figure 7 shows the deployment the design targets.

Table 6: Nodes of the production deployment

Node	Runs	Notes
Customer or administrator device	Storefront, console	Any current browser.
Control host	<code>swarmops-core</code>	Ubuntu 24.04 with systemd. Reads the database DSN and keys from protected files.

Node	Runs	Notes
Database server	The swarmops schema	MariaDB 11.4 LTS or MySQL 8.4 LTS. Backed up with <code>mysqldump --single-transaction</code> ; the data key is kept separately.
Edge manager node	Agent, Traefik, applications	Labelled <code>nim.edge=true</code> ; publishes ports 80 and 443.
Worker nodes	Agent, applications	One or more.

7.2 Environment used for verification

Figure 8 shows the environment in which the system was actually run and verified on 15 September 2026. It differs from production in the ways listed after the figure.

- Core and the agent ran as processes on a macOS laptop rather than as systemd services on Ubuntu.
- The Swarm had a single node, the laptop’s Docker Engine, which served as both manager and edge.
- The agent was a source build stamped with version v0.22.0, the version of this code base.
- The browser reached the applications through the Vite development server rather than through Core’s embedded files.
- Certificates were not issued, because the gateway had no public hostname.

8 References

- [1] S. Brown, “The C4 model for visualising software architecture.” [Online]. Available: <https://c4model.com>. Accessed: Sep. 2026.
- [2] R. Nordmann and contributors, “C4-PlantUML.” [Online]. Available: <https://github.com/plantuml-stdlib/C4-PlantUML>. Accessed: Sep. 2026.
- [3] PlantUML. [Online]. Available: <https://plantuml.com>. Accessed: Sep. 2026.

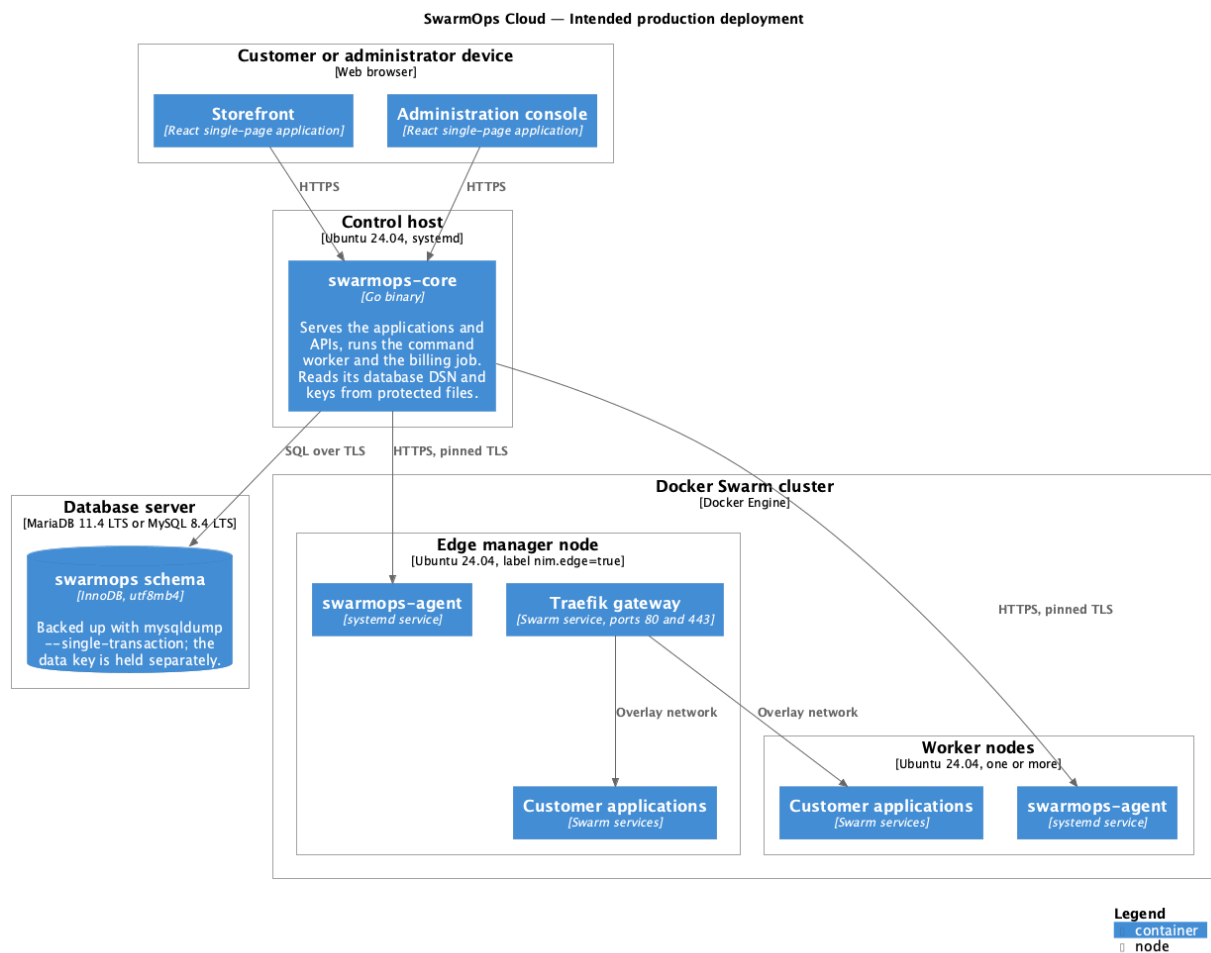
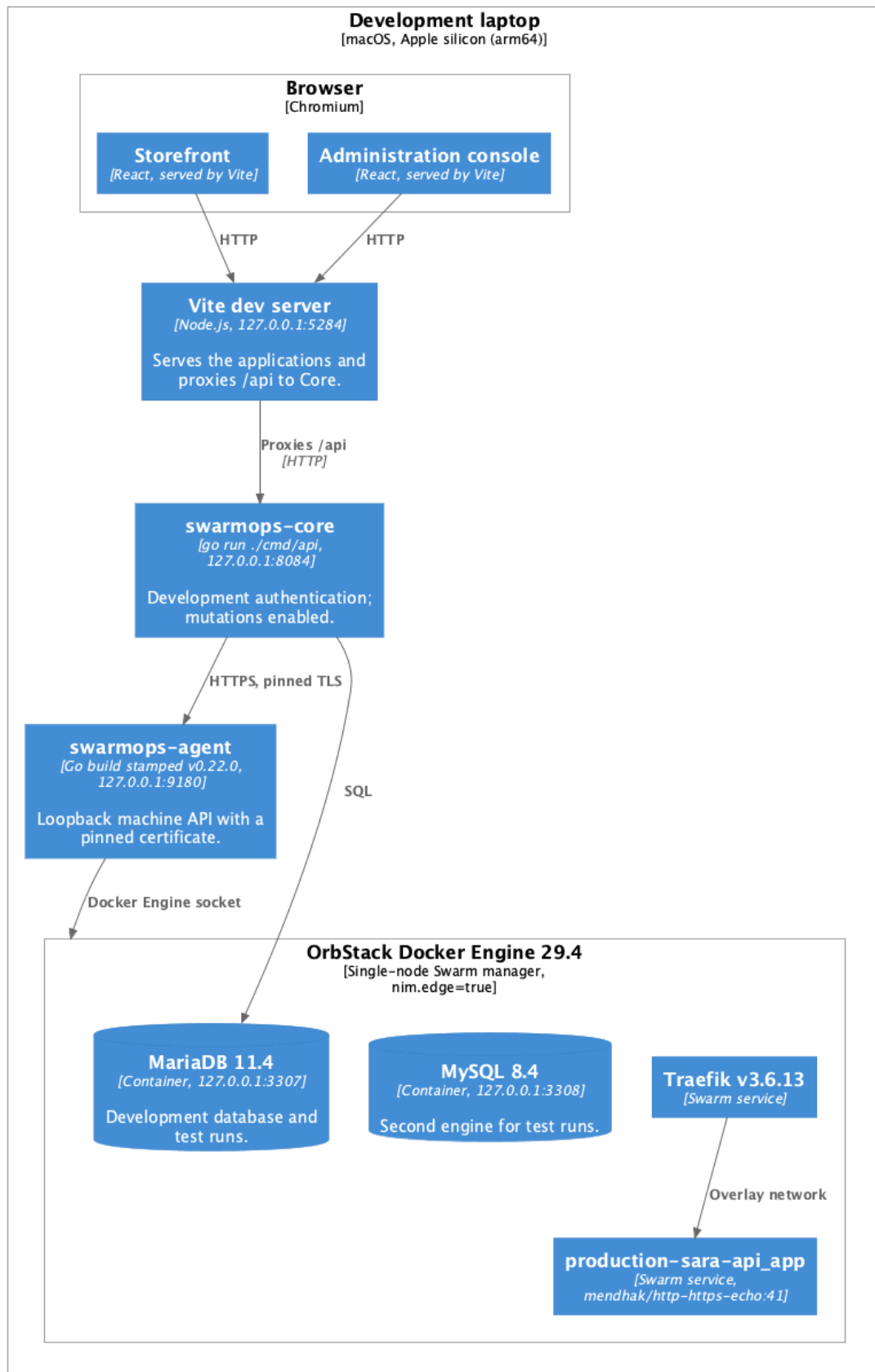


Figure 7: Intended production deployment

SwarmOps Cloud — Environment used for verification (15 September 2026)



Legend
 container
 node

Figure 8: Environment used for verification